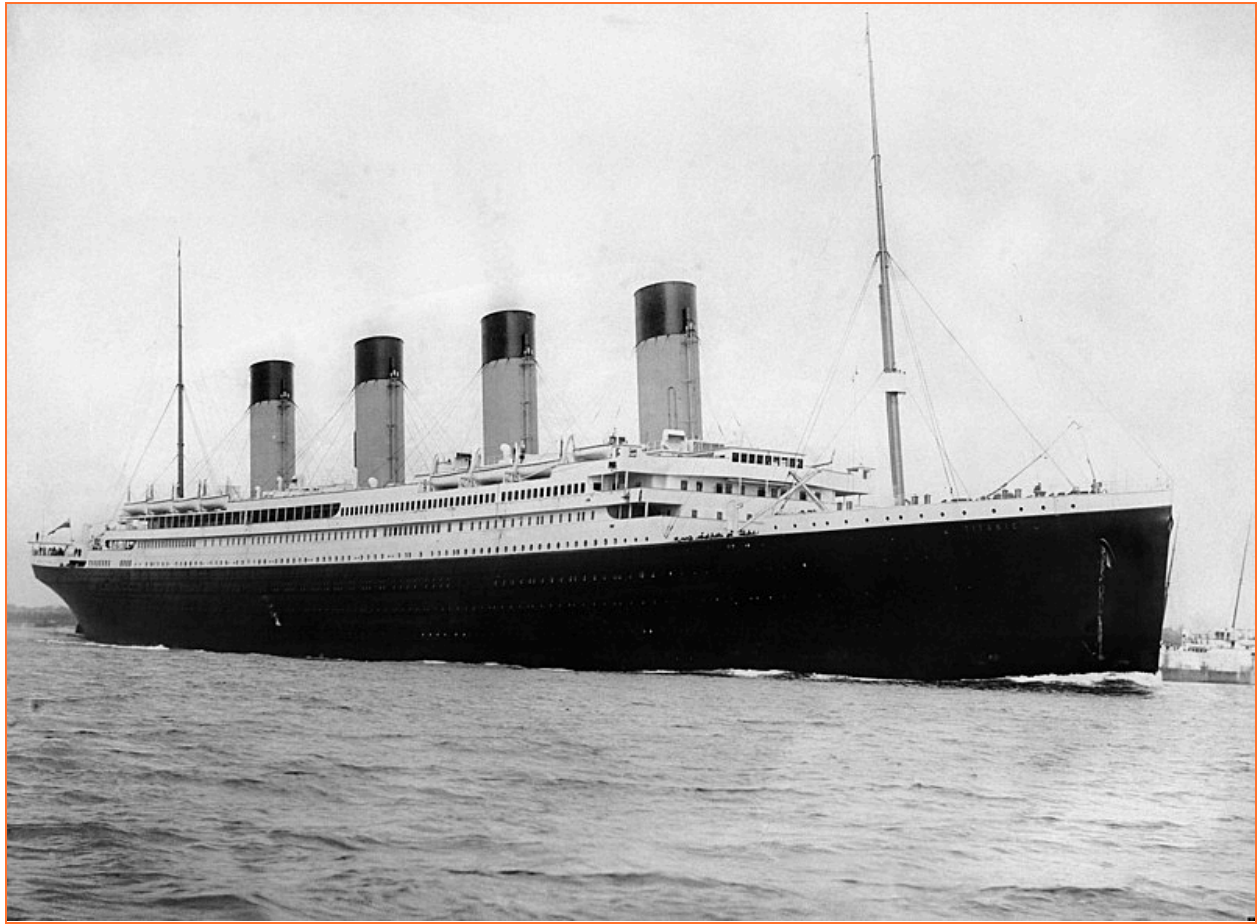




Rapport de projet

L211 : Machine Learning from disaster

Encadrant : Paul Boniol

Hongxiang Lin

Melissa Merabet

Mathieu Antonopoulos

Timothé Miel



2022 - 2023



Référence du projet	L2I1
Intitulé	'Machine Learning from disaster'
Date de rédaction du document	05/23
Auteurs	Mathieu Antonopoulos Melissa Merabet Timothé Miel Hongxiang Lin
Confidentialité	UFR Maths-Info de l'université Paris-Cité
Date de soumission	16/05/23

Sommaire

1. Préface.....	4
2. Introduction.....	5
2.1. Contexte.....	5
2.2. Objectifs.....	6
2.3. Motivations.....	6
3. Ressources.....	7
4. Outils de développement.....	8
5. Organisation et répartition.....	9
6. Analyse des données.....	10
6.1. Analyse de la qualité des variables.....	10
6.2. Analyse des facteurs.....	14
6.3. Prétraitement des données.....	19
7. Les modèles de machine learning.....	21
7.1. Les différents modèles.....	21
7.2. Précision des modèles.....	24
7.3. Améliorations et ouverture.....	26
8. Application Web.....	27
8.1. Bilinguisme.....	28
8.2. Visualisation des Données.....	28
8.3. Prédiction Personnalisée.....	29
8.4. Analyse des Données.....	29
8.5. Défis rencontrés et Solutions.....	29
8.6. Contribution de l'application web.....	31
9. Remerciements.....	33
10. Conclusion.....	34
11. Références:.....	36

1. Préface

Dans le cadre de notre deuxième année de licence d'informatique et applications à l'université de Paris-Cité, il nous est proposé de travailler en groupe sur un projet qui s'étendra sur une période de 4 mois. Ce projet vise à mettre en pratique nos connaissances et capacités face à des problématiques réelles, constituant pour la plupart d'entre nous une première expérience de projet informatique. De plus, ce projet offre l'opportunité de découvrir de nouvelles notions et outils informatiques.

Le document suivant constitue le rapport final de notre projet intitulé : “L2I1: *Machine Learning from disaster*”. Il expose en détail l'ensemble des étapes nécessaires à la réalisation du projet. Il retrace ainsi son déroulement à travers les hypothèses, déductions et autres réflexions émises par le groupe. Notre objectif est de présenter au lecteur un aperçu complet et clair de notre cheminement de pensée, permettant ainsi une pleine compréhension du projet et de son développement.

Dans un premier temps, nous détaillons le contexte dans lequel s'inscrit le projet, Nous aborderons ensuite les problématiques et objectifs avant de parler de la conception et du développement. Nous ferons également le point sur les difficultés rencontrées et l'organisation du groupe.

2. Introduction

2.1. Contexte

Le naufrage du *Titanic* demeure l'un des événements tragiques les plus célèbres de l'histoire. Le 15 avril 1912, le *RMS Titanic*, largement considéré comme un navire "insubmersible", a sombré lors de son voyage inaugural après avoir heurté un iceberg. Malheureusement, la capacité limitée des canots de sauvetage a entraîné la perte de 1502 des 2224 passagers et membres d'équipage. Bien que la survie des individus présente une part de chance liées à de nombreux facteurs aléatoires, des observations suggèrent que certains groupes de personnes ont plus de chances de survivre que d'autres. Le *Titanic* fait encore parler de lui plus d'un siècle après son naufrage, il s'immortalise à travers les différentes adaptations cinématographiques et références littéraires. En effet, cet événement surréaliste attise encore à ce jour la curiosité et se positionne comme la référence de désastre maritime.

Ainsi, notre équipe se met au défi de répondre à la question suivante : "Quels groupes de personnes ont le plus de chances de survivre au naufrage du *Titanic*?".

Le projet consiste à développer un modèle prédictif par machine learning* (Apprentissage automatique), en utilisant les données relatives aux passagers du *Titanic*, afin de prédire qui aurait le plus de chances de survivre au naufrage. Les données des passagers comprennent des informations telles que leur nom, leur âge, leur sex, leur classe socio-économique, leur statut de survie, etc. Le produit final se décompose en deux parties distinctes : la première partie est le modèle prédictif codé en *Python*, accompagné d'une documentation détaillée, explicitant le fonctionnement du modèle et les analyses effectuées. La seconde partie étant une interface web permettant une visualisation interactive des résultats.

2.2. Objectifs

L'objectif principal du projet est de créer un programme *python* permettant de renvoyer avec une précision maximum, une prédiction de survie pour un passager donné (1 si le passager survit 0 sinon). Un des objectifs principaux est également d'obtenir le meilleur classement possible dans le concours *Kaggle 'Titanic: Machine Learning from Disaster'*. En effet, notre projet s'articule autour d'un concours en ligne, ouvert à tous et proposé par la plateforme *Kaggle*. Il s'agit d'une plateforme proposant des concours d'apprentissage en sciences de données. Le site *Kaggle* propose également un classement des meilleures équipes en fonction de la précision de leur modèle. Le concours compte plus de 16 000 équipes participantes à ce jour. En début de projet, nous avons fixé pour objectif la création d'un modèle ayant un taux de précision de 0.8. Nous avons également voulu rendre notre modèle utilisable et disponible en ligne. Nous avons donc fixé comme objectif supplémentaire, considéré comme facultatif au début du projet, la création d'une interface permettant l'utilisation du modèle.

2.3. Motivations

Tous les membres du groupe ont été attirés par ce projet, il offre une opportunité de découvrir et de pratiquer le Machine Learning à travers une expérience ludique sous forme de concours. Nous sommes tous intéressés par le machine learning et ses applications, et ce projet permet de nous initier à la discipline à travers la création de notre premier modèle. Au début du projet, nous étions tous de parfaits débutants en la matière, n'ayant jamais étudié ces notions auparavant. Grâce à ce projet, et avec le soutien et le suivi de notre encadrant, nous en avons énormément appris sur le sujet. Pour certains d'entre nous, il a même permis d'ajuster notre poursuite d'études en fonction de nos intérêts et compétences nouvellement découverts. La composante 'concours' nous a placé dans un cadre compétitif, ce qui nous a motivé à donner le meilleur pour atteindre le meilleur classement.

3. Ressources

Pour débiter le projet, on disposait de certaines ressources sur la plateforme de concours Kaggle. On nous fournit des sets de données et des explications de bases afin d'amorcer l'analyse des données. Ainsi on dispose de deux sets de données :

- 'train.csv' : Contient les informations de 891 passagers (Statut de survie, nom, âge, sex, classe socio-économique, prix du billet, nombre de frères et sœurs présents à bord, nombre de parents ou enfants, numéro de cabine et port d'embarcation).
- 'test.csv' : Contient les informations de 418 passagers supplémentaires. Cependant, on ne dispose pas du statut de survie pour ces passagers.

Voici un tableau récapitulatif du contenu de chaque sets de données :

TRAIN.CSV	TEST.CSV
Survived : Survie du passager : 0 ou 1	Ø
Name : nom, prénom (et titre honorifique)	
Sex : genre ('male' ou 'female')	
Âge	
SibSp : nombre de frères et sœurs, époux inclus	
Parch : nombre de parents et d'enfants	
Ticket : numéro du billet (ex : PC 17758)	
Fare : prix du billet (en \$)	
Cabin : numéro de cabine (ex : C105)	
Embarked : port d'embarcation (S: Southampton, C : Cherbourg, Q : Queenstown)	

4. Outils de développement

On utilise le langage de programmation *Python* dans sa version 3.1. Il s'agit du langage le plus courant pour la création de modèle de machine learning et intègre de nombreuses bibliothèques utiles à l'analyse et l'observation de données.



On utilise la plateforme web interactive *Jupyter* pour créer un Notebook. Cette plateforme implémente le langage *Python* et permet donc de manipuler et visualiser les données. De plus *Jupyter* nous permet de plus facilement mettre en commun notre travail.

Pour le développement de l'application web, on a utilisé *Streamlit*, un framework open-source Python spécialement conçu pour les ingénieurs en machine learning et l'analyse de données. Ce framework permet de créer des applications web qui intègrent des modèles de machine learning et des outils de visualisation de données.



Scikit-learn est une bibliothèque Python pour le machine learning. Celle-ci regroupe de nombreuses fonctions et algorithmes indispensables à la création et l'entraînement d'un modèle. Nous avons également utilisé les librairies *matplotlib*, *seaborn* et *NumPy* pour visualiser les données. Enfin la librairie *pandas* nous a permis de manipuler et gérer les données.

Anaconda est une distribution libre de langages de programmation *Python* appliqué conçu pour la science des données et le machine learning. *Anaconda* nous permet donc d'utiliser *Jupyter* et d'avoir un environnement et un IDE* adaptés au projet.



5. Organisation et répartition

Etant donné que ce projet est une initiation au machine learning, nous avons partagé le travail de sorte à ce que tous les membres du groupe puissent s'essayer à toutes les tâches. Le but étant que chacun maîtrise l'ensemble des notions abordées.

Voici la répartition des principales tâches sur les 12 semaines de conception et développement :

Phase d'exploration	Semaine 1 et 2
Analyse des données	Semaine 2 à 6
Construction du modèle	Semaine 6 à 10
Documentation et Notebook	Semaine 9 et 10
Développement de l'application Web	Semaine 9 à 11
Rédaction du rapport et préparation en vue de la soutenance	Semaine 12

6. Analyse des données

6.1. Analyse de la qualité des variables

Nous avons analysé les données en termes de complétude et disponibilité, nous avons repéré les informations manquantes dans chaque colonne de notre base de données (Âge, Embarked, Cabin). Nous avons également mappé les valeurs de quelques colonnes, c'est-à-dire, nous avons modifié les types des données afin de les gérer efficacement et facilement (Embarked, Cabin, Sex). Enfin, nous avons ajouté 4 nouvelles variables que nous calculons à partir d'autres features (Civility, TicketFrequency, Family, Alone).

★ Âge :

Il s'agit d'une colonne importante pour la prédiction de la survie ou non d'un passager, cependant il manque 177 valeurs dans cette colonne qui représente 20% des données du dataframe. On devra donc la compléter.

Pour celà, nous avons émis 3 hypothèses:

→ Hypothèse 1 :

Remplacer les valeurs manquantes par la médiane ou la moyenne des valeurs existantes qui sont respectivement de 32 et 31.1. Mais, comme la moyenne est influencée par les valeurs extrêmes alors nous avons opté pour la médiane qui est une meilleure mesure de la tendance centrale lorsque les distributions sont asymétriques. On a atteint un score de 0.806 avec ce choix.

→ Hypothèse 2:

Compléter les valeurs manquantes par la médiane en fonction de la classe sociale qui correspond à la colonne "Pclass", car les passagers de la première classe ont tendance à être plus âgés. Les valeurs d'âge obtenues sont:

- Classe 1: 38 ans
- Classe 2: 30 ans
- Classe 3: 24 ans

Avec cette hypothèse nous avons réalisé un score de 0.79.

→ Hypothèse 3:

Remplir les lacunes par la médiane en fonction du sexe qui correspond à la colonne "Sex". Les valeurs d'âges obtenues sont:

- Femmes : 27 ans
- Hommes : 29 ans

Avec cette hypothèse nous avons obtenu un score de 0.77.

On peut constater une différence de score 0.03 par rapport au meilleur score (hypothèse 1), cela revient au fait que cette méthode crée une corrélation entre l'âge et le sexe et donc double l'importance de cette feature sans apporter d'informations supplémentaires.

Nous avons donc conclu que la complétion de la colonne "Age" par la médiane est la meilleure solution.

★ Embarked :

Étant donné qu'il n'y a que deux valeurs manquantes dans cette colonne, nous les avons complétées par la valeur la plus fréquente qui est "S".

Nous avons mappé les valeurs de la colonne "Embarked" comme suit :

- 1 représente la valeur "C"
- 2 représente la valeur "S"
- 3 représente la valeur "Q"

★ Cabin :

Nous avons constaté qu'il manque 687 valeurs dans la colonne "cabin" qui équivaut à 77%. Nous avons juste mis "null" dans les cases manquantes car nous n'avons pas assez d'informations pour déterminer un lien entre les données d'un passager et sa cabine.

Après avoir effectué des recherches sur le Titanic, nous avons trouvé le schéma représentatif de la structure de ce dernier, illustré dans l'image (1). Par ce schéma, on constate que les passagers de la première classe habitaient les cabines situées sur les ponts A, B ou encore C, et ceux de la troisième classe étaient principalement logés sur les ponts F ou G. Afin de simplifier la donnée, nous conservons uniquement la lettre du numéro de cabine.

Nous avons par la suite changé les valeurs de A-T par 1 à 8 et les valeurs manquantes portent désormais le numéro 9.

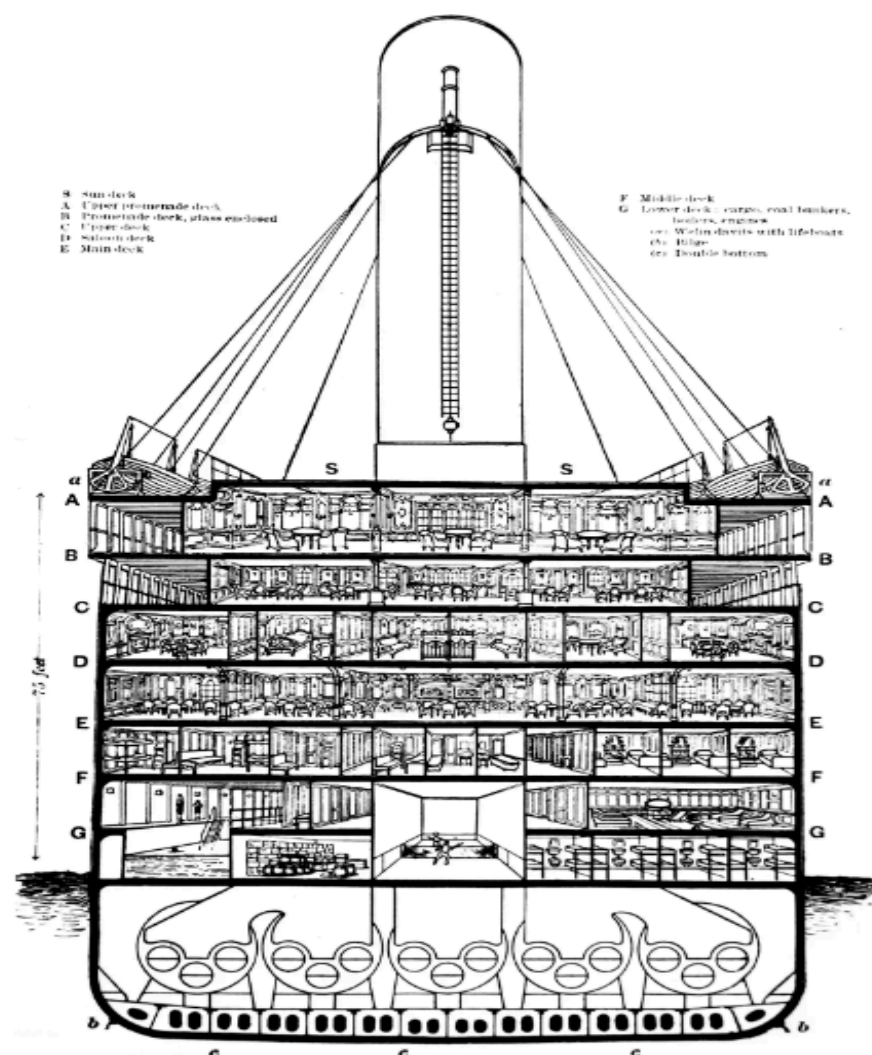


Image 1: Plan des cabines du Titanic (coupe frontale)

★ Sex :

Nous avons mappé les valeurs de la colonne "Sex" comme suit:

- 0 représente les hommes
- 1 représente les femmes

★ Civility :

On a observé qu'il y a plusieurs civilités dans la base de données, les plus fréquentes sont "Mr", "Miss" et "Mrs". De plus, il y a plusieurs autres civilités qui ne sont pas très répandues comme "Ms" et "Sir". Nous avons donc pris en compte que les civilités portées

par plus de 20 passagers et avons considéré que les autres civilités sont des exceptions pouvant mener à des erreurs de prédiction.

Nous avons également modifié quelques valeurs comme suit:

- "Mme" est représenté par "Mrs"
- "Mlle" est représenté par "Miss"
- "Sir" est représenté par "Mr"

Nous avons donc obtenu les statistiques suivantes:

- "Master" est compté 40 fois.
- "Miss" est compté 184 fois.
- "Mr" est compté 518 fois.
- "Mrs" est compté 126 fois.
- "nul" est compté 23 fois.

Nous avons ensuite créé de nouvelles colonnes pour obtenir plus d'informations sur les passagers :

★ TicketFrequency

Cette colonne représente la fréquence d'apparition d'un numéro de ticket spécifique parmi les passagers. Cette colonne va nous permettre d'identifier les groupes de passagers qui partagent le même numéro de ticket ce qui peut être utile pour capturer les relations entre les passagers. Il est à noter que la fréquence du numéro de ticket peut influencer la survie d'un passager, car ceux qui voyagent ensemble peuvent potentiellement avoir le même statut de survie.

★ Family

De la même façon que "TicketFrequency", la colonne "Family" va nous permettre de regrouper les membres d'une famille à l'aide des colonnes "Parch" (enfant/parent) et "Sibsp" (frère/soeur).

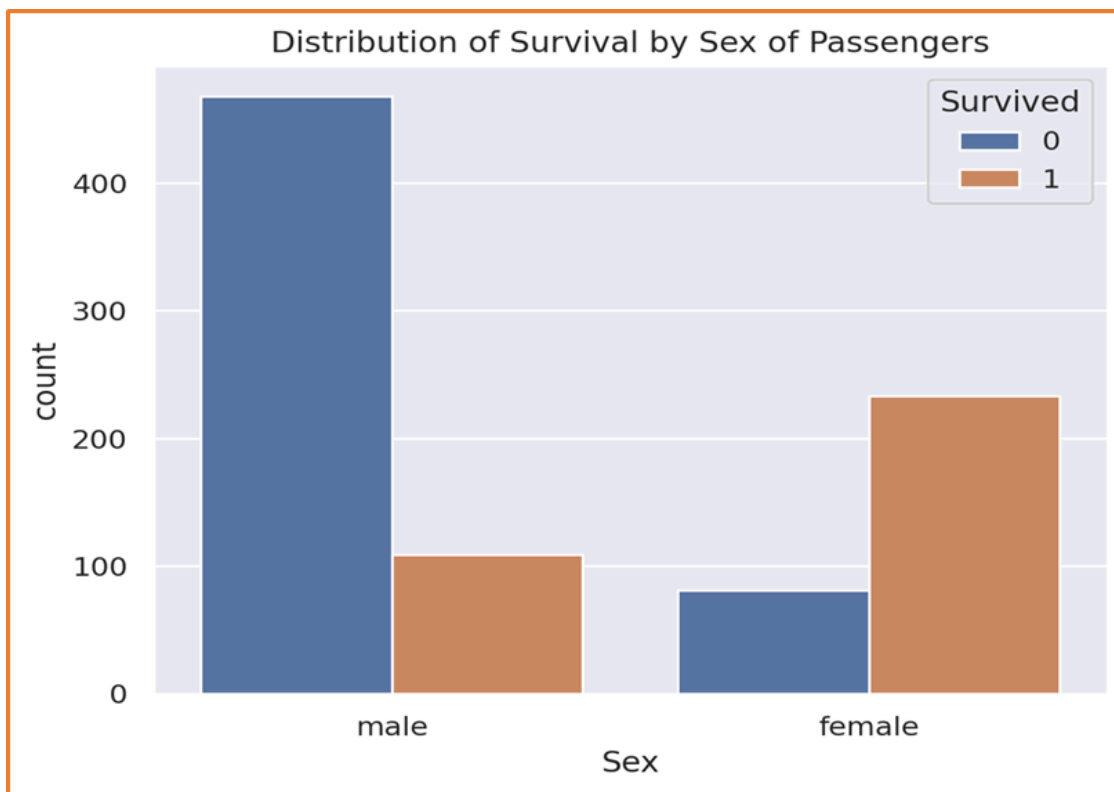
★ Alone

Cette colonne nous fait savoir si un passager a voyagé seul ou en famille, peut être intéressant pour observer l'impact de la présence ou non de la famille au bord du Titanic sur la survie du passager.

6.2. Analyse des facteurs

L'analyse des facteurs consiste à explorer les données, trouver les liens existant entre les différentes variables de notre base de données et déduire les informations statistiques qui permettront d'optimiser le modèle prédictif. Diverses techniques de visualisation ont été utilisées pour mieux comprendre les relations entre les données et les variables. Parmi les histogrammes que nous avons tracés, nous citons:

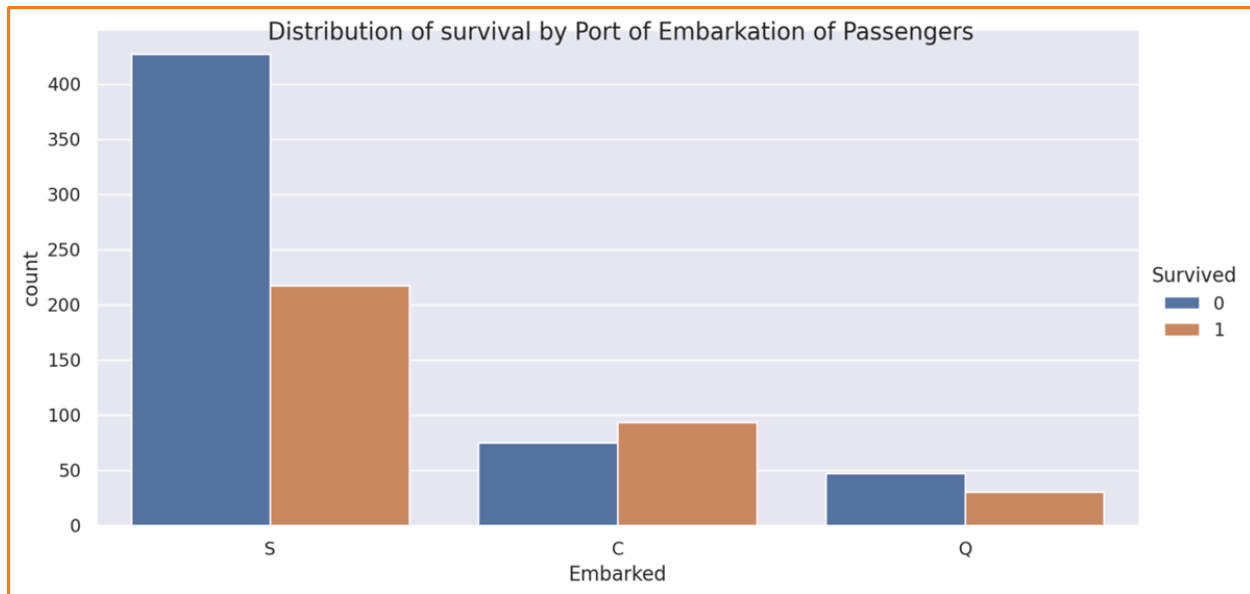
➤ **Le nombre de survivants en fonction du sexe:**



Histogramme 1: La distribution des survivants en fonction du sexe

Nous remarquons par cet histogramme qu'environ 100 hommes ont survécu parmi 550 hommes, et qu'une centaine de femmes sont décédées contre environ 250 survivantes. Plus de 2/3 des survivants sont de sexe féminin.

➤ **Le nombre de survivants en fonction du port d'embarquement:**

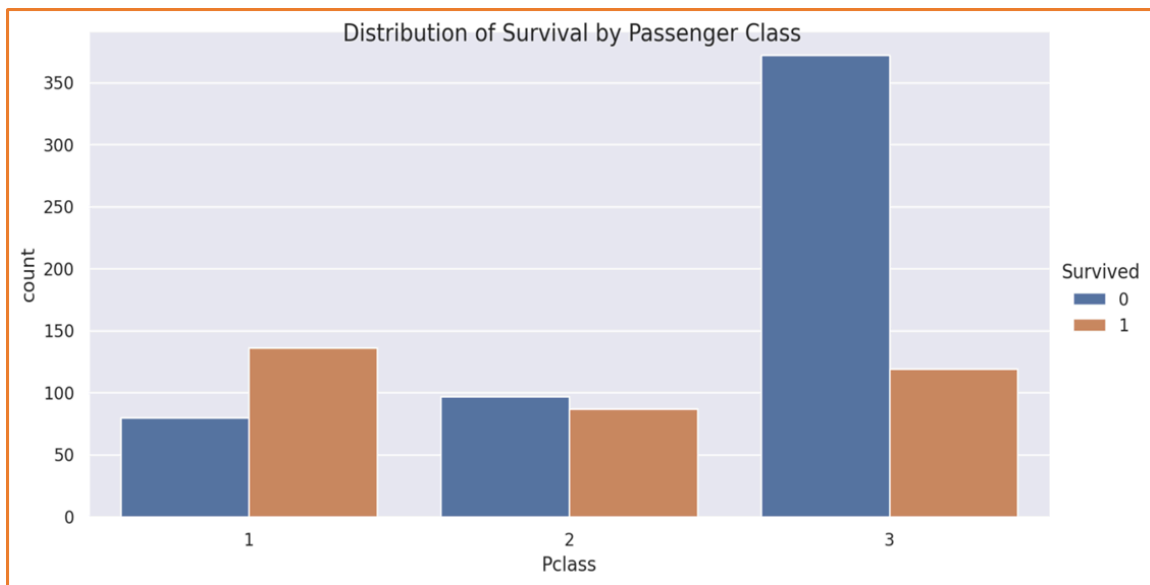


Histogramme 2: La distribution des survivants en fonction du port d'embarquement

Par cet histogramme, nous réalisons que:

- Pour les passagers ayant embarqué depuis le port de Cherbourg, il y a eu plus de survivants que de morts (environ 75 morts et 90 survivants);
- Pour ceux ayant embarqué depuis le port de Southampton il y a eu plus de morts que de survivants (plus de 400 morts et environ 230 survivants);
- Pour ceux ayant embarqué depuis le port de Queenstown il y a eu plus de morts que de survivants (environ 50 morts et 40 survivants);
- Alors, ce sont les passagers qui ont embarqué depuis le port de Cherbourg qui ont le plus survécu, et le pourcentage de décès est plus élevé pour ceux qui ont embarqué depuis Southampton. Cela peut être lié à la classe des passagers embarquant dans chaque port (hypothèse).

➤ **Le nombre de survivants en fonction de la classe sociale:**

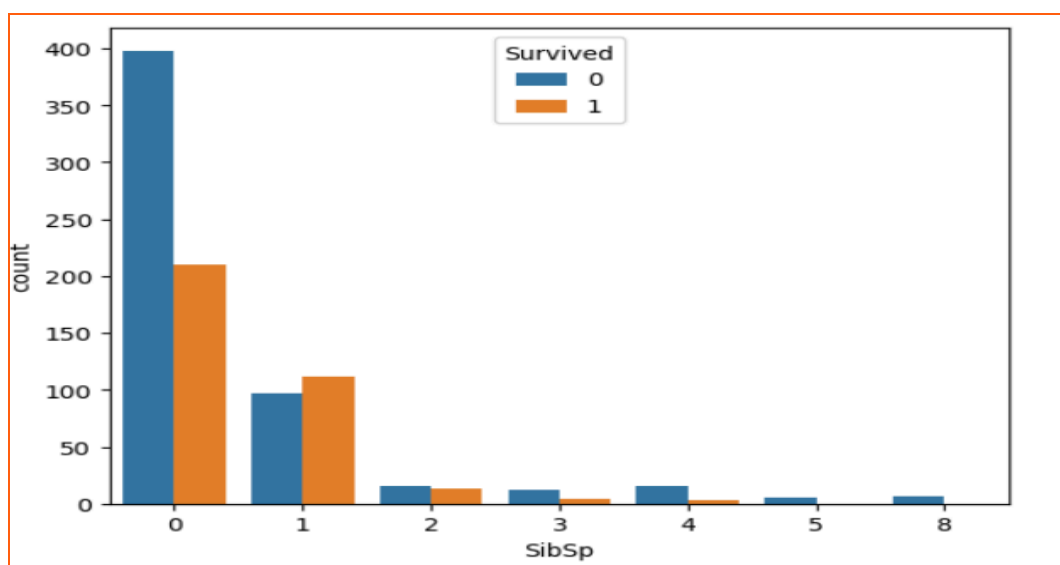


Histogramme 3: La distribution des survivants en fonction de la classe sociale

Nous remarquons que:

- Pour les passagers de la 1ère classe, il y a largement plus de survivants que de morts (environ 140 survivants et 80 morts);
- Pour ceux de la 2ème classe, il y a eu plus de morts que de survivants (environ 85 survivants et 100 morts);
- Pour la 3ème classe sociale, il y a beaucoup plus de morts que de survivants (environ 120 survivants et plus de 370 morts);
- Ainsi nous avons conclu que ce sont les passagers de la 1ère classe qui ont le plus survécu, et le pourcentage de décès est plus élevé pour les passagers de la 3ème classe.

➤ **Le nombre de survivants en fonction du nombre de frères et soeurs:**

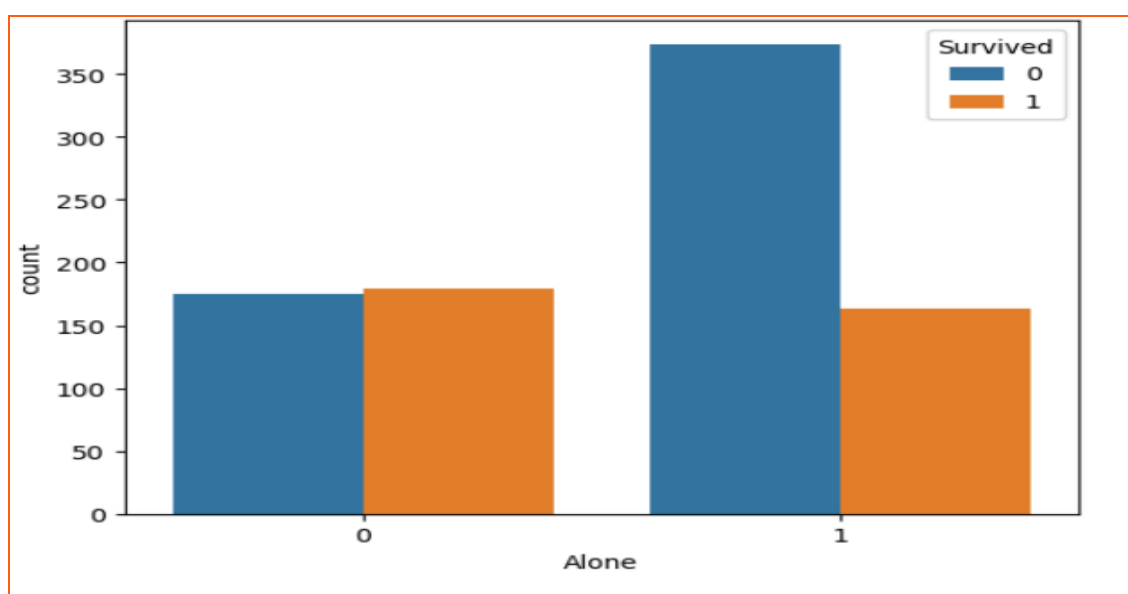


Histogramme 4: la distribution des survivants en fonction du nombre des frères et soeurs

Ce graphique permet de dire que:

- La majorité des passagers n'avaient pas de frères et soeurs au bord du Titanic;
- Il y a un très faible nombre de personnes qui ont plus de 5 frères et soeurs à bord;
- Les passagers ayant embarqué avec un(e) frère/soeur ont plus de chance de survivre que ceux qui en avaient plus ou ceux qui ont embarqué seuls.

➤ **Le nombre de survivants en fonction de leur statut d'accompagnement:**

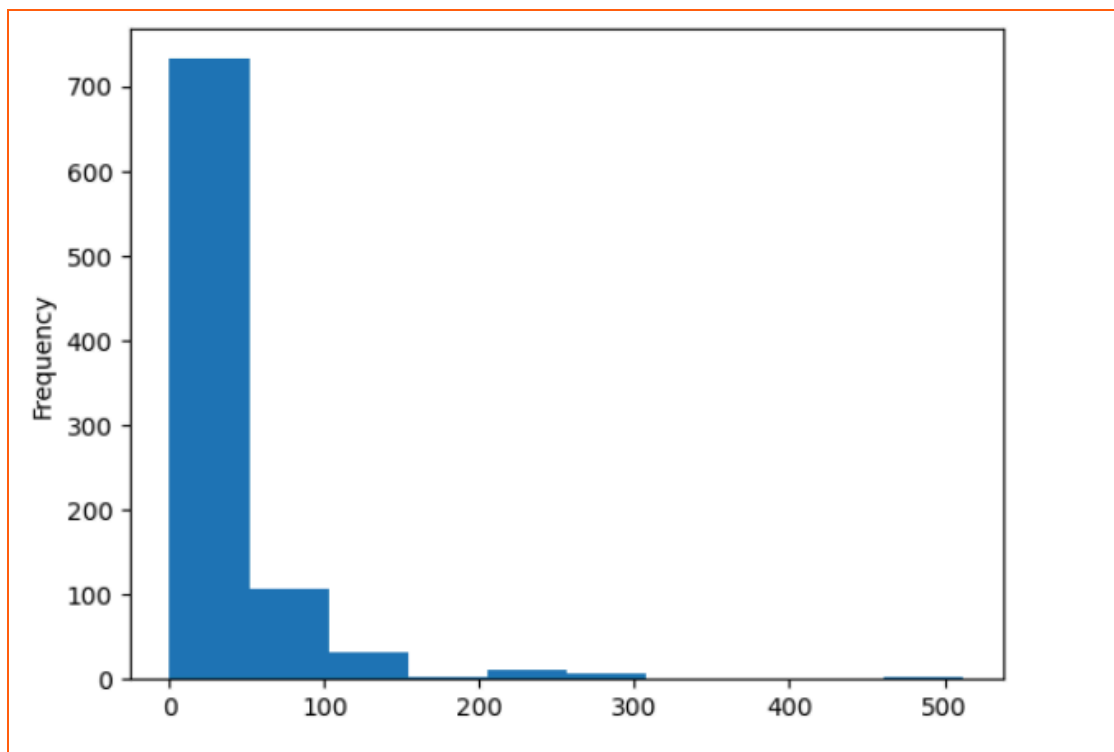


Histogramme 5: La distribution des survivants en fonction de leur statut d'accompagnement

En observant cet histogramme nous avons constaté que:

- Pour les passagers qui étaient seuls à bord, il y a eu un peu près le même nombre de morts que de survivants (environ 175);
- Pour les passagers qui étaient accompagnés à bord, il y a eu plus de morts que de survivants (environ 170 survivants et plus de 360 morts);
- Nous pouvons donc dire que le taux de survie est plus important pour ceux qui étaient seuls.

➤ **La distribution des tarifs des tickets:**

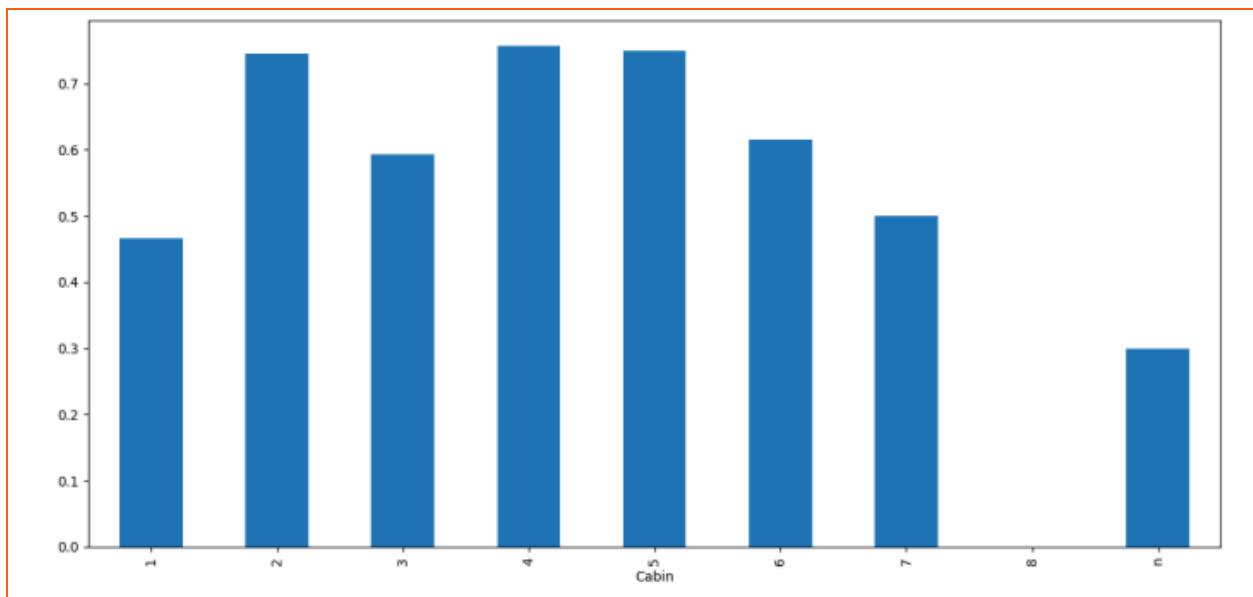


Histogramme 6: La distribution des tarifs des tickets.

L'histogramme montre que:

- Plus de 700 passagers ont pris des billets à moins de 50;
- Quelques billets ont coûté 500.

➤ **La moyenne des taux de survie des passagers en fonction de leur cabine:**



Histogramme 7: La moyenne des taux de survie des passagers en fonction de leur cabine

Nous pouvons observer que:

- La moyenne des passagers vivants dans les cabines B,D et E est supérieure à 0.7;
- La moyenne des passagers vivants dont la valeur de "Cabin" est manquante est de 0.3;

6.3. Prétraitement des données

1) **Standardisation des données**

La standardisation (ou normalisation) des caractéristiques autour du centre avec un écart type de 1 est importante car nous comparons des mesures qui ont des unités différentes (age, monnaie, nombre de famille,...). Les variables mesurées à différentes échelles contribuent de manière différente à l'analyse, telle qu'une variable qui prend de grandes valeurs l'emportera sur celle qui prend de petites valeurs ce qui peut générer de fausses informations sur la corrélation des features.

2) **La corrélation entre les différentes features**

Nous avons pu déterminer les relations linéaires possibles entre les variables grâce à la matrice de corrélation. Cela nous a permis d'identifier les variables pouvant avoir un impact significatif sur la survie ou non d'un passager.

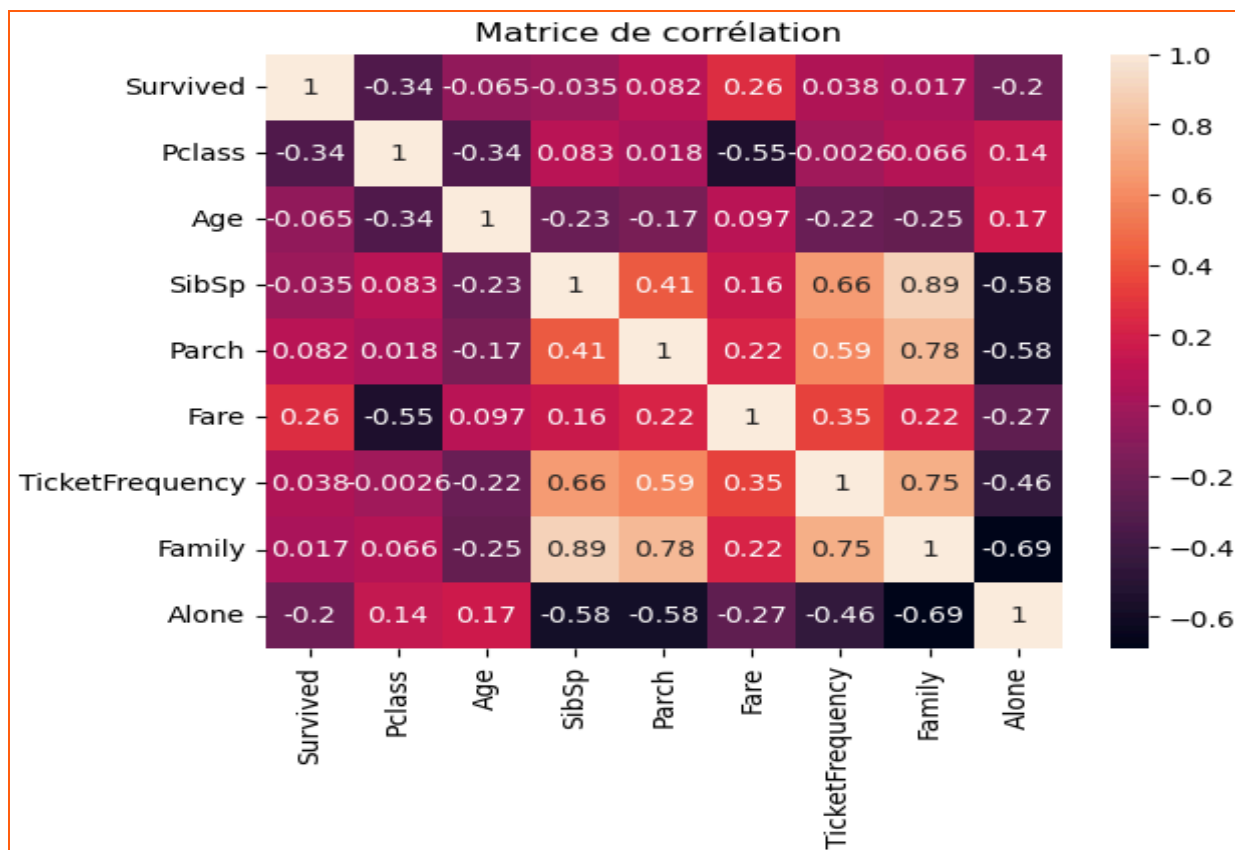


Image 2: La matrice de corrélation

- Nous pouvons voir qu'il y a une corrélation négative entre "Pclass" et la colonne "Survived". Ceci dit que les personnes de la première classe avaient plus de chance de survie que ceux de classes inférieures;
- On constate également que la colonne "Fare" corrèle positivement avec "Survived". Cela suggère que ceux qui ont payé leurs billets plus chers avaient plus de chance de survivre;
- La colonne "Alone" présente aussi une corrélation négative avec "Survived". Cela indique que les passagers qui ont voyagé seuls ont plus de chance de survie que ceux qui étaient accompagnés.

3) Suppression des features moins importantes:

Après avoir testé plusieurs fois les algorithmes sur les différentes features, nous avons conclu qu'il faut supprimer ces features : Name / Civility / Cabin / TicketFrequency / Family / Alone.

7. Les modèles de machine learning

7.1. Les différents modèles

Une fois les données prétraitées, notre encadrant nous a suggéré d'amorcer la construction du modèle, et une nouvelle question se pose : quel modèle de prédiction serait le plus adéquat afin d'obtenir le meilleur résultat sur Kaggle ?

Pour ce faire nous avons utilisé la librairie SciKit-Learn (scikit-learn.org) qui possède de nombreuses fonctions de modèles prédictifs en python, et également de nombreuses autres fonctions utiles en machine learning (pour faciliter le preprocessing, comparer différents modèles, et bien d'autres). Pour tester les modèles et leur efficacité, nous étions contraints d'utiliser le fichier train.csv, étant donné que le fichier test.csv ne contient pas la colonne « Survived ». Pour ce faire, nous avons divisé le fichier train.csv en 4 différents « fichiers » :

- X_train contenant les données de 70% des passagers sans la colonne « Survived »
- Y_train contenant la colonne « Survived » de ces 70% correspondants
- X_test contenant les données des 30% restants, sans la colonne « Survived »
- Y_test contenant la colonne « Survived » de ces 30% restants

Nous avons choisi la répartition 70%/30% car elle permet de s'entraîner de s'entraîner sur un assez large échantillon de données tout en conservant une quantité significative de données tests. Une fois train.csv divisé de cette manière, nous pouvions désormais entraîner les différents modèles à notre disposition, en entraînant les modèles sur les données X_train et en les corrélant à la colonne « Survived » de Y_train, ils effectueront des prédictions en se basant sur les données de X_test et nous comparerons (à l'aide d'une fonction) Y_test avec la prédiction effectuée par le modèle. Nous aurons désormais un score de précision qui nous permettra de savoir l'efficacité d'un modèle donné, en fonction de ses paramètres et des features qu'il prend en compte.

Nous avons testé les modèles suivants :

- Decision Tree Classifier
- Linear Support Vector Classifier (LSV)
- Logistic Regression
- Gaussian Naive Bayes (GaussianNB)
- K-neighbors Classifier
- AdaBoost Classifier

- Random Forest Classifier
- Multi-Layer Perceptron Classifier

Nous avons choisi des modèles assez courants et accessibles. En effet, il existe des modèles plus performants sur des services payants. Nous nous sommes donc limités aux modèles de la bibliothèque libre 'Scikit Learn'.

Pour certains de ces modèles, les tests de précisions furent aisés à réaliser de par le fait qu'il y ait peu d'hyper-paramètres à faire varier, comme GaussianNB, Logistic Regression et LSV, où l'efficacité du modèle dépend peu des paramètres utilisés. Pour ces modèles basés sur des algorithmes et des fonctions mathématiques, il semblerait qu'il est préférable d'avoir le plus de features possibles afin d'obtenir le meilleur résultat. Cependant, on ne conserve pas toutes les features créées (notamment celles sur les étages du bateaux : A, B, C...) car elles semblaient faire baisser la précision des modèles. Ce phénomène est appelé « l'overfitting » et il décrit le fait qu'un modèle sur-apprend les données, qu'il interprète trop et donne trop de poids à certaines données. Ce qui a pour effet certes d'obtenir un bon résultat de comparaison avec Y_{test} . Mais lorsque l'on soumet nos résultats sur Kaggle, c'est-à-dire que l'on fait un test d'efficacité du modèle sur test.csv, on obtient une précision bien moindre. On peut expliquer ce phénomène par la variance des données. Le modèle aura considéré certains cas extrêmement précis de 'train.csv' qui ne se trouvent pas dans 'test.csv'. De manière générale, on a pu observer une baisse de précision du modèle sur les données de 'test.csv' par rapport au test des données d'entraînements. On perdait 0.03 de précision en moyenne jusqu'à 0.07 pour le modèle ADA-boost. Par conséquent, tout le long du projet, nous n'obtiendrons jamais la même précision entre le résultat prédit par le modèle sur Y_{test} que par celui prédit sur test.csv.

Pour revenir aux modèles, le modèle K-neighbors Classifier possède un fonctionnement assez intuitif, expliquer son fonctionnement est plus aisé « en 2 dimensions » : Sur un plan où des billes rouges et bleues sont réparties, lorsque l'on ajoute une nouvelle bille, K-neighbors Classifier va décider la couleur de cette bille en fonction des K voisins les plus proches, K un entier; si K vaut 3 et que les billes les plus proches sont 2 billes rouges et une bille bleue, alors la nouvelle bille sera rouge. Les billes sont donc nos passagers, leur couleur correspond à leur survie, et le plan représente les différentes données et leurs valeurs possibles. Le paramètre que nous avons décidé de modifier était donc celui qui fait varier le nombre de voisins.

Nous avons aussi utilisé le modèle Decision Tree Classifier, son mode de fonctionnement est le plus aisé à comprendre car c'est aussi comme cela que le cerveau décide de fonctionner pour résoudre certains problèmes : on prend un passager puis on regarde, est-ce un homme ou une femme ? Si c'est une femme, quel âge a-t-elle ? Si c'est un

homme, est-il venu avec des enfants ? Et ainsi de suite, question par question le modèle se rapproche peu à peu du résultat et donne enfin sa prédiction. Pour les paramètres, nous avons principalement modifié celui influant sur le nombre de «questions» (max_depth, le nombre de questions va changer la profondeur maximum de l'arbre).

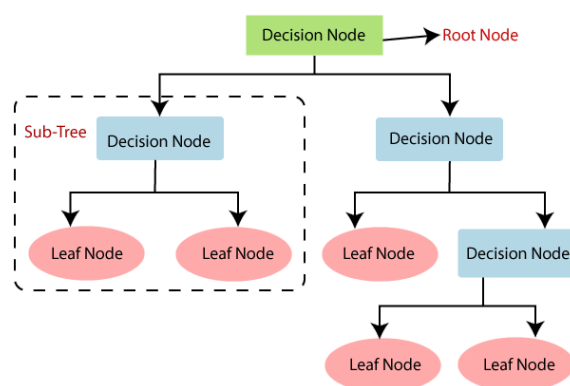


Schéma du fonctionnement du modèle Decision tree Classifier

Passons au modèle Random Forest Classifier. Il s'agit d'un modèle ensembliste se basant sur les arbres de décisions. Il va en effet créer plusieurs arbres (nombre correspondant au paramètre n_estimators) chacun d'une profondeur maximum donnée (correspondant à max_depth). Tous ces arbres seront un peu différents les uns des autres, et enfin le modèle donnera sa prédiction finale en effectuant un vote à la majorité entre tous ses arbres. Ici, le couple de paramètres fait que les possibilités à tester sont nombreuses, nous avons pu malgré cela, accélérer un peu le processus en utilisant une autre fonction fournie par Scikit-Learn : GridSearchCV (voir 'Précision des modèles').

Notre encadrant nous a ensuite suggéré d'essayer un modèle supplémentaire en vue d'améliorer nos résultats : AdaBoost Classifier

AdaBoost est un modèle dit de « boosting » : il va commencer par des estimateurs faibles (ici Decision Tree avec une profondeur maximum de 1) et faire des estimations. À chaque «tout» Adaboost va créer un nouvel estimateur qui va s'ajuster et « se concentrer » là où l'estimateur précédent a eu faux et ainsi de suite (le nombre d'itérations dépend du paramètre n_estimators). De ce fait, plus il y a d'itérations, plus le modèle sera précis. AdaBoost est un modèle qui fonctionne particulièrement bien sur Y_test mais lorsque l'on réalise la soumission sur Kaggle, il perd énormément d'efficacité (probablement encore une fois de l'overfitting).

7.2. Précision des modèles

GridSearchCV est une fonction de scikit learn qui permet de tester en un appel de fonction, plusieurs combinaisons de paramètres, en créant deux ensembles de valeurs de paramètres. GridSearchCV nous retournera la combinaison de paramètres la plus précise. Cela a pu être utile pour Random Forest Classifier et d'autres modèles pour trouver la meilleure combinaison de hyperparamètres. De plus, Random Forest possède un attribut (`features_importances_`) qui permet de déterminer quels features le modèle utilise le plus afin de réaliser ses prédictions, on a donc pu en déduire que pour éviter l'overfitting et donc permettre au modèle de « se concentrer » sur l'essentiel, les features à garder étaient celles considérées comme les plus importantes selon lui (dans l'ordre : le sexe, le prix du billet, l'âge, la classe, le nombre de frères et sœurs et de conjoint.e.s, le nombre de parents/enfants, et le port d'embarcation). Random Forest Classifier est le modèle le plus performant que nous ayons pu utiliser, c'est également celui qui nous a permis d'obtenir notre place élevée au concours sur Kaggle.

Voici les résultats obtenus en utilisant 'feature_importance_' sur Random Forest :

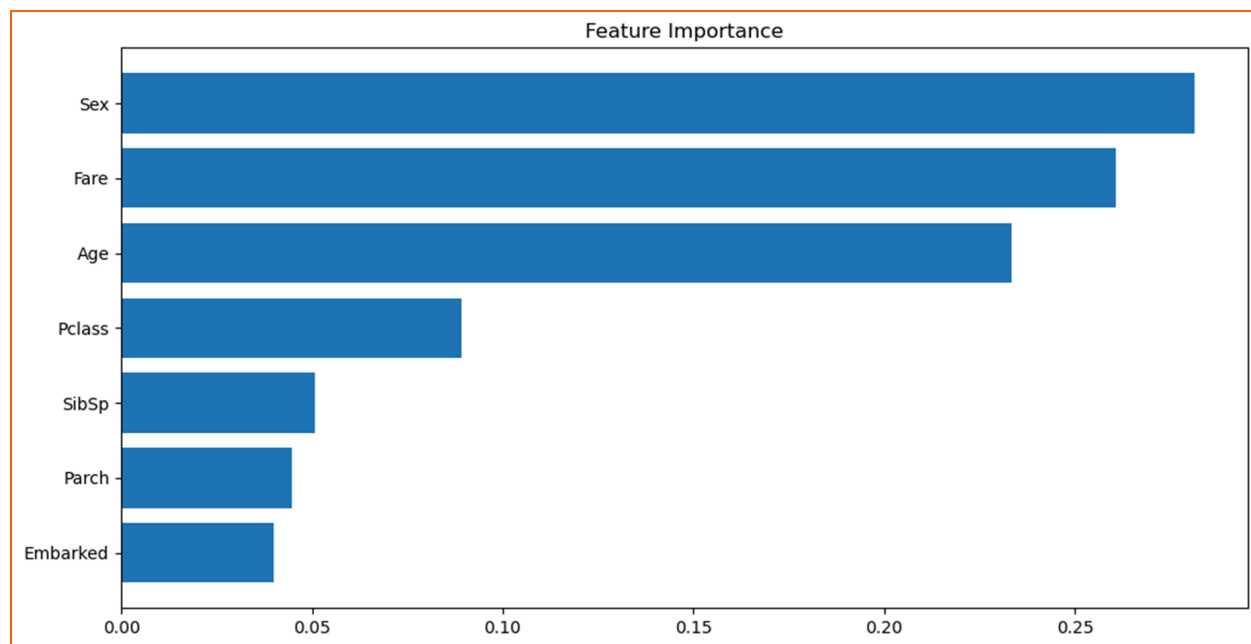


Image 2: Graphe sur l'importance des features

Voici un résumé des modèles utilisés, les hyperparamètres permettant d'obtenir les meilleurs résultats et de la précision du modèle sur les données 'X_test' :

Modèle	Paramètres	Score
Random Forest	n_estimators=160,max_depth=13	0.839552
K Nearest Neighbors	n_neighbors = 8	0.813433
AdaBoost Classifier	n_estimators=40, random_state=0	0.802239
Decision Tree	Valeurs par défaut	0.787313
Gaussian Naive Bayes	Valeurs par défaut	0.787313
Linear Support Vector	Valeurs par défaut	0.776119
Logistic Regression	Valeurs par défaut	0.772388

Nous obtenons le meilleur score pour le modèle 'Random Forest'. Il s'agit du modèle utilisé pour la soumission finale.

7.3. Améliorations et ouverture

Pour aller plus loin, on a utilisé la méthode dite de voting :

La méthode de vote est une méthode qui permet, comme son nom l'indique, de faire voter les modèles entre eux. Afin de contrebalancer les faiblesses des uns avec les forces des autres. Nous avons pris les prédictions de tous les modèles et, pour chaque passager, nous additionnons les réponses. Si la majorité des modèles prédisent la survie, le passager sera déclaré survivant, dans le cas contraire, il sera déclaré mort. C'est donc la majorité des modèles qui conserve la prédiction finale. En cas d'égalité, nous avons convenu de conserver la prédiction du modèle Random Forest, le plus performant. Étonnement, le résultat obtenu n'était pas plus élevé, et restait donc inférieur à la précision de Random Forest seul. Cela s'explique par le fait que certains modèles donnent les mêmes prédictions. En effet, certains modèles ont un fonctionnement similaire (Decision Tree et Random Forest par exemple) ce qui peut créer des erreurs sur les mêmes données et ainsi fausser la prédiction générale.

- Précision (sur Y_{test}) du vote : 0.7

Cependant certaines pistes auraient pu être explorées, en effet Scikit-Learn propose une fonction de vote qui permet d'accorder plus de poids à certains modèles (soft-voting). On pourrait ainsi éviter de donner trop de poids aux modèles similaires. De plus, nous avons pu observer que certains modèles sont meilleurs que d'autres pour les vrais-survécus alors que d'autres sont meilleurs pour les vrais-décédés. (À l'aide d'une matrice de confusion), ce qui permettrait de construire un vrai modèle de vote en fonction des forces de chaque modèle. Cependant, ces méthodes suggèrent une étude approfondie de chaque combinaison de modèles possibles. Les contraintes de temps ne nous ont pas permis d'aller plus loin dans cette direction.

8. Application Web

Lien : <https://projetl2i1.streamlit.app/> ou QR code :



Introduction : Dans le cadre de notre projet, nous avons développé une application web de visualisation des données en utilisant Streamlit, un outil de création d'application basé sur Python. Cette application comprend quatre fonctionnalités majeures, notamment la traduction en deux langues, la visualisation de données, la prédiction personnalisée et l'analyse des données. Cette partie du rapport se concentrera sur les fonctionnalités implémentées dans l'application et leur contribution à notre objectif global.

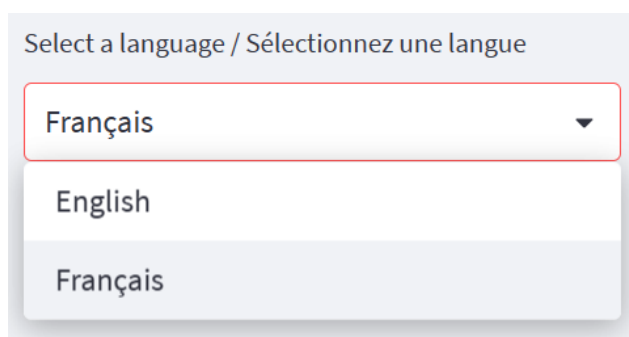
Importation des bibliothèques nécessaires:

Pour le développement de cette application, nous avons utilisé plusieurs bibliothèques Python. Ci-dessous, les bibliothèques importées :

- **streamlit** : une bibliothèque open-source utilisée pour créer rapidement des applications Web de visualisation de données.
- **pandas** : une bibliothèque de manipulation et d'analyse de données.
- **numpy** : une bibliothèque pour le calcul scientifique.
- **seaborn** et **matplotlib** : les bibliothèques de visualisation de données.
- **sklearn.ensemble** et **sklearn.model_selection** : les modules de la bibliothèque Scikit-learn utilisés pour le Machine Learning. Nous avons utilisé **RandomForestClassifier** pour la prédiction et **train_test_split** pour la division des données.

8.1. Bilinguisme

Pour assurer une accessibilité maximale, nous avons intégré la prise en charge des langues anglaise et française dans notre application. Cela signifie que tous les éléments de contenu, y compris les menus, les boutons et les instructions, sont disponibles dans les deux langues. Cette fonctionnalité permet aux utilisateurs de choisir la langue qui leur convient le mieux, facilitant ainsi leur navigation sur le site même s'ils ne comprennent pas le français.



Select a language / Sélectionnez une langue

Français ▼

English

Français

8.2. Visualisation des Données

Dans l'onglet "Datasets", nous avons inclus les ensembles de données "train.csv" et "test.csv". Les utilisateurs peuvent librement parcourir ces ensembles de données pour explorer les informations détaillées sur les passagers du Titanic. Cette fonctionnalité vise à offrir aux utilisateurs une compréhension approfondie des données sous-jacentes et à encourager leur engagement avec le contenu.

train.csv (Notre dataset d'entraînement):

	PassengerId	Survived	Pclass	Name	Sex	Age	S
0	1	0	3	Braund, Mr. Owen Harris	male	22	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	
2	3	1	3	Heikkinen, Miss. Laina	female	26	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	
4	5	0	3	Allen, Mr. William Henry	male	35	
5	6	0	3	Moran, Mr. James	male	None	
6	7	0	1	McCarthy, Mr. Timothy J	male	54	
7	8	0	3	Palsson, Master. Gosta Leonard	male	2	
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27	
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14	

8.3. Prédiction Personnalisée

Dans l'onglet "Would you survive", nous avons mis en place une fonctionnalité permettant aux utilisateurs de créer des informations d'un passager personnalisé. Nous avons utilisé un modèle de forêt aléatoire (Random Forest) en nous basant sur les meilleurs paramètres que notre équipe a obtenus lors de notre participation au défi Kaggle (0.806 top 1.2% du classement). Les utilisateurs peuvent choisir différentes valeurs pour les caractéristiques (features) du passager, ce qui leur permet de générer un profil personnalisé. Notre modèle prédit ensuite si ce passager spécifique aurait survécu ou non lors du naufrage du Titanic. Cette fonctionnalité interactive apporte une expérience ludique aux utilisateurs et leur permet de mieux comprendre comment les différentes caractéristiques influencent les résultats de survie.

Project L211: Machine Learning from Disaster

Veuillez définir les informations suivantes pour prédire le résultat de survie de ce passager:

Prenom du passager

Passenger Class

1

Sex

male

Age

25

Nombre total de frères et soeurs et de conjoint du passager

0

0

8

Nombre total de parents et d'enfants du passager

0

0

10

Prix du billet

50.00

0.00

600.00

Port d'embarquement

C

Résultat: non survivant

8.4. Analyse des Données

Dans l'onglet "Data analysis", nous avons regroupé les histogrammes clés issus de notre analyse initiale des ensembles de données. Cela permet aux utilisateurs de visualiser plus facilement les résultats de nos analyses et d'avoir une compréhension plus intuitive des tendances et des relations entre les différentes variables.

8.5. Défis rencontrés et Solutions

Au cours du processus de développement de cette application, nous avons rencontré plusieurs défis.

★ mise en place du support multilingue:

Pour rendre l'application accessible à un public plus large, nous avons décidé d'inclure deux des langues les plus parlées au monde : l'anglais et le français. En utilisant un dictionnaire Python pour stocker le contenu dans chaque langue, nous avons pu garder notre code organisé et facilement modifiable. Ici, nous avons généré deux dictionnaires "english" et "french".

- Choix de la langue par l'utilisateur: Pour permettre à l'utilisateur de choisir la langue de l'application, nous avons utilisé un selectbox dans la barre latérale de Streamlit. En fonction de la langue sélectionnée, la variable `selected_language` est mise à jour. Cette variable est ensuite utilisée pour accéder au contenu approprié dans notre dictionnaire.
- Accès au contenu de langue sélectionnée: Chaque fois que nous avons besoin d'afficher du texte à l'utilisateur, nous utilisons simplement `[selected_language]` pour accéder au contenu dans la langue appropriée.

Cette approche nous a permis de garder notre code propre et organisé, tout en rendant l'application accessible à un public international.

★ Gestion des onglets:

Un autre défi important que nous avons rencontré était de gérer les différents onglets de l'application. Pour résoudre ce problème, nous avons utilisé la fonction `st.tabs` de Streamlit qui nous a permis de créer plusieurs onglets dans notre application.

Dans le code, nous avons créé d'abord les onglets en utilisant la fonction `st.tabs`. Les onglets sont créés en fonction de la langue choisie par l'utilisateur. Par exemple, si l'utilisateur a choisi l'anglais comme langue, les titres des onglets seront en anglais. Si l'utilisateur a choisi le français, les titres des onglets seront en français.

Ensuite, nous utilisons la fonction `with` pour gérer le contenu affiché dans chaque onglet. Par exemple, lorsque l'utilisateur clique sur l'onglet "Wiki", la fonction `show_wiki()` est appelée et le contenu de cet onglet est affiché.

Cette approche nous a permis de créer une interface utilisateur conviviale qui est facile à naviguer, tout en offrant un accès facile à toutes les fonctionnalités de notre application.

★ Prédiction de la survie d'un passager personnalisé:

L'une des fonctionnalités clés de notre application est la possibilité pour l'utilisateur de créer un profil de passager personnalisé et de prédire si ce passager aurait survécu au naufrage du Titanic. Pour réaliser cela, nous avons utilisé un certain nombre de widgets Streamlit pour recueillir les données du passager, y compris `text_input`, `selectbox` et `slider`.

Dans le code, nous recueillons les informations suivantes sur le passager : nom, classe de billet "Pclass", sexe, âge, nombre de frères et sœurs / conjoints à bord "Sibsp", nombre de parents / enfants à bord "Parch", tarif du billet "Fare" et port d'embarquement "Embarked".

Les utilisateurs ont la possibilité de saisir leurs propres valeurs pour ces caractéristiques, ce qui rend cette fonctionnalité à la fois interactive et personnelle. Par exemple, ils peuvent utiliser le widget slider pour sélectionner un âge entre 0 et 100 ans, ou utiliser le widget selectbox pour choisir le sexe du passager.

Une fois que toutes ces informations ont été saisies, nous utilisons le modèle Random Forest avec les meilleurs paramètres que notre équipe a obtenus lors de notre participation au défi Kaggle (0.806 top 1.2% du classement) pour prédire si le passager aurait survécu ou non. Cette prédiction est ensuite affichée à l'utilisateur.

Cette fonctionnalité illustre bien l'interactivité et la personnalisation que Streamlit permet dans la construction d'applications Web, et elle a été un élément central de notre travail sur cette partie.

8.6. Contribution de l'application web

C'est une application web innovante, accessible gratuitement en ligne depuis n'importe quelle plateforme, y compris mobile et PC. Elle offre une interface intuitive qui permet aux utilisateurs d'entrer des données personnelles et de recevoir instantanément une prédiction, basée sur notre modèle de Machine Learning, de leur probabilité de survie s'ils avaient été passagers à bord du Titanic. Cette approche ludique fait de l'interaction avec notre modèle une expérience attrayante et éducative.

L'utilisateur peut tester le modèle avec une variété de données, permettant ainsi d'identifier les facteurs qui ont influencé les chances de survie lors de ce désastre historique. Ceci offre un aperçu précieux de la logique sous-jacente de notre modèle, tout en mettant en évidence les réalités sociales et économiques de l'époque. Par exemple, la classe du billet, le sexe et l'âge sont tous des facteurs significatifs, reflétant les inégalités sociales et économiques qui ont joué un rôle dans la détermination des survivants du naufrage du Titanic.

En plus de cette fonctionnalité de prédiction personnalisée, notre application offre également un aperçu détaillé de nos ensembles de données via l'onglet "Datasets". Elle présente une vue d'ensemble de notre analyse de données dans l'onglet "Data analysis", et fournit des informations détaillées sur le projet dans l'onglet "Wiki". Chacun de ces onglets a été conçu avec soin pour faciliter l'accessibilité et l'interaction, renforçant ainsi l'engagement des utilisateurs avec notre travail.



Enfin, l'application se distingue par son support bilingue anglais-français, augmentant ainsi sa lisibilité à une audience plus large et globale.

9. Remerciements

La réalisation de ce projet a été possible grâce à la collaboration de plusieurs personnes à qui nous voudrions témoigner notre gratitude.

Nous tenons à exprimer notre reconnaissance à notre encadrant M Boniol pour sa patience, sa disponibilité et surtout ses judicieux conseils qui ont contribué à alimenter notre réflexion.

Nous désirons remercier également tous les chercheurs et informaticiens qui ont participé à l'amélioration de la librairie Scikit-Learn, la plateforme Streamlit ainsi que les différentes bibliothèques de Python comme Seaborn et Matplotlib. Sans leurs efforts, nous n'aurions pas pu bénéficier de ces puissantes ressources incroyables pour l'apprentissage automatique.

Enfin, nous remercions tous nos professeurs de l'UFR mathématique et informatique qui nous ont fourni les outils nécessaires à la réalisation de ce projet.

10. Conclusion

Dans ce projet, après avoir analysé la qualité des données et extrait les informations et liens entre les différentes variables à notre disposition nous avons pu déterminer les features les plus influentes sur la survie des passagers qui sont le sexe, le prix du billet et l'âge en premières positions puis viennent la classe sociale et le statut d'accompagnement des passagers, ceci dit que le prix du billet a plus d'influence que la classe sociale du passager. Après avoir utilisé les différents modèles de la bibliothèque Scikit-Learn, et avoir effectué plusieurs tests sur les paramètres qui améliorent le score de chacun des modèles nous sommes parvenus à réaliser un modèle de prédiction avec un score de 80.6% avec l'algorithme Random Forest qui prend comme paramètres `max_depth` et `n_estimators` qui prennent respectivement les valeurs 13 et 160.

Après cette étude, nous pouvons répondre à notre problématique étant "Quels groupes de personnes ont le plus de chances de survivre au naufrage du *Titanic*?". Ce sont principalement les femmes, les passagers qui ont payé leurs billets plus chers, ceux de la première classe sociale et ceux qui ont embarqué seuls qui ont plus de chances de survivre.

Nous pouvons donc dire que nous avons atteint les objectifs fixés ; nous avons élaboré un modèle de prédiction avec un score de 80.6% ce qui a permis d'avoir un classement parmi les tops 1.2% sur le concours Kaggle. Nous avons trouvé les variables les plus influentes sur les chances de survie des passagers. Nous avons également fourni une interface web qui permet de visualiser les données pour mieux comprendre les caractéristiques des passagers du Titanic et permet également à l'utilisateur de faire des prédictions sur la survie d'un passager en entrant ses caractéristiques.

Ce projet peut cependant être amélioré pour augmenter le score de notre modèle de prédiction. Plusieurs pistes sont proposées qui par faute de temps n'auront pu être suivies:

- Exploitation de plus de features: En utilisant d'autres features comme la colonne "Cabin" qui porte sur la cabine des passagers à bord du Titanic comme il s'est penché vers l'avant. Ou encore, d'essayer d'extraire plus d'informations sur la colonne "Embarked" pour connaître les raisons pour lesquelles le taux de survie est élevé pour ceux qui ont embarqué de Cherbourg et le taux de décès est beaucoup plus élevé pour ceux qui ont embarqué depuis Southampton. Il est aussi tentant d'exploiter la colonne "Civility", qui porte sur la civilité de chaque passager, par hypothèse que les passagers de différentes civilités peuvent être traités différemment lors de l'évacuation.
- Utilisation de modèles plus avancés: On pourrait se servir du modèle MLPClassifier qui utilise un réseau de neurones, il est capable d'apprendre à partir des données d'entraînements.

- Utilisation d'ensembles de modèles: En expérimentant le soft-voting qui consiste à combiner les prédictions de plusieurs modèles qui peuvent avoir des performances ou des biais différents pour produire une prédiction finale qui fusionne les forces de chaque modèle.

En conclusion, ce projet nous a initié au machine learning et ses applications. Nous avons élaboré un premier modèle, nous avons manipulé plusieurs types de modèles aux fonctionnements très différents. Cela nous a permis de mieux comprendre les différentes formes d'apprentissage automatique. Nous avons aussi fait de l'analyse de données, indispensable à l'élaboration du modèle. Cela nous a permis d'acquérir une certaine méthodologie d'analyse, exploration des données, hypothèse, visualisation et enfin conclusion.

11. Références

- [Our Documentation | Python.org](#)
- [Anaconda | The World's Most Popular Data Science Platform](#)
- [Image 1: Plan des cabines du Titanic](#)
- [Matplotlib — Visualization with Python](#)
- [seaborn: statistical data visualization — seaborn 0.12.2 documentation \(pydata.org\)](#)
- [Delft Stack](#)
- [scikit-learn: machine learning in Python — scikit-learn 1.2.2 documentation](#)
- [Streamlit • A faster way to build and share data apps](#)